



**Międzyuniwersyteckie
Centrum Informatyzacji**
<http://muci.edu.pl>

USOS 4.1

Rejestracja do grup

Silnik do realizacji giełdy

Krzysztof Ciebiera
Uniwersytet Warszawski

Styczeń 2010

Spis treści

1	Wstęp	2
2	Wejście i wyjście	3
2.1	Uwagi ogólne	3
2.2	Struktura pliku wejściowego	3
2.2.1	Zajęcia	3
2.2.2	Grupy związane	4
2.2.3	Studenci	5
2.2.4	Przykładowy plik wejściowy	6
2.3	Struktura pliku wyjściowego	7
2.3.1	Przykładowy plik wyjściowy	7
2.4	Raport	7
3	Opis działania programu	8
3.1	Algorytm	8

Rozdział 1

Wstęp

W tym dokumencie opisujemy silnik do realizacji giełdy procesie rejestracji do grup. Dokument jest podzielony następująco:

Rozdział 2 opisuje format wejścia i wyjścia programu, sposób jego uruchamiania oraz opis błędów sygnalizowanych przez program.

Rozdział 3 opisuje budowę programu jak również algorytm i funkcję celu stosowaną do oceny jakości znajdowanych przez program rozwiązań.

Rozdział 2

Wejście i wyjście

2.1 Uwagi ogólne

- Program stosuje kodowanie UTF-8.
- Jeśli nie podajemy typu danych, to mamy na myśli dziewięciocyfrowe liczby całkowite.
- Tablice zapisujemy w stylu pythonowym czyli w nawiasach kwadratowych po-oddzielane przecinkami. Przykład `[[a,b,c],[d,e,f],[g,h]]`
- Białe znaki w obrębie wiersza są dopuszczalne.
- Puste wiersze są dopuszczalne.
- Od znaku `#` zaczyna się komentarz, który trwa do końca wiersza. Komentarz jest pomijany przez program.

2.2 Struktura pliku wejściowego

Plik składa się logicznie z dwóch części. Pierwsza część pliku opisuje zajęcia i grupy, a druga opisuje preferencje osób. Części następują bezpośrednio po sobie.

2.2.1 Zajęcia

Pierwszy wiersz pliku zawiera liczbę zajęć k . Potem następuje opis k zajęć. Pojedynczy opis zajęć składa się z:

- liczby całkowitej, która jest jednoznacznym identyfikatorem zajęć (można tu użyć np. pola `zaj_cyk_id`, czyli klucza zajęć w tablicy `dz_zajecia_cykli`, które jednoznacznie identyfikuje zajęcia),

- zawartego w cudzysłowach opisu czytelnego dla człowieka¹,
- tablicy opisującej poszczególne grupy.

Każda grupa jest opisywana przez tablicę 4-elementową zawierającą:

- numer grupy (unikatowy w obrębie zajęć),
- limit miejsc w grupie,
- liczbę zajętych miejsc w grupie,
- tablicę terminów, w których odbywają się zajęcia tej grupy, każdy z elementów tej tablicy jest tablicą zawierającą 3 liczby o następujących intuicyjnych znaczeniach:
 - dzień tygodnia (poniedziałek = 1, wtorek = 2, ..., niedziela = 7),
 - pora rozpoczęcia w formacie hhmm, czyli 9:00 to 900, 20:09 to 2009,
 - pora zakończenia w formacie hhmm.

Liczba wolnych miejsc

W pliku wejściowym podawane są limit miejsc i liczba miejsc zajętych w grupie. Na podstawie tych dwóch informacji program oblicza liczbę wolnych miejsc w grupie. Jeśli liczba zajętych miejsc jest większa niż limit grupy, to program uznaje, że limit tej grupy jest równy liczbie zajętych miejsc.

Giełda traktuje limit miejsc i liczbę zajętych miejsc odmiennie niż rejestracja do grup. Algorytm zastosowany w giełdzie *nigdy nie przekracza limitów*.

Uwagi ogólne dotyczące formatu opisu zajęć

Część informacji w pliku wejściowym nie jest potrzebna do działania programu. Dane są nadmiarowe, ponieważ chcemy zachować maksymalną kompatybilności pliku wejściowego z plikiem tworzonym na potrzeby rejestracji do grup.

2.2.2 Grupy związane

Po opisach poszczególnych zajęć następują opisy grup związanych. Pierwszy wiersz tej części pliku zawiera liczbę m będącą liczbą grup związanych. Każdy z następnych m wierszy opisuje pojedynczą grupę związaną. Opis grupy związanej, składa się z:

- numeru zajęć,

¹ Program nie oczekuje żadnej konkretnej formy tego opisu jedynie wymaga, aby opis ten był pojedynczym napisem (może to być np. kod cyklu dydaktycznego, kod przedmiotu i typ zajęć). W raportach opis ten będzie się pojawiał jako nazwa zajęć.

- numeru grupy,
- numeru zajęć związanych,
- tablicy zawierającej numery grup związanych.

Informacje o grupach związanych nie są wykorzystywane przez program. Pakiety tworzone przez studentów muszą zawierać w sobie grupy związane.

2.2.3 Studenci

Po opisie zajęć następują opisy preferencji studentów. Pierwszy wiersz tej części pliku zawiera liczbę n będącą liczbą studentów. Następne n wierszy to opisy poszczególnych studentów. Opis studenta składa się z:

- numeru studenta,
- zawartego w cudzysłowach opisu studenta czytelnego dla człowieka,
- wagi studenta (co najwyżej dziewięciocyfrowa liczba całkowita),
- tablicy zajęć i grup, do których jest zapisany student,
- tablicy pakietów na które chce się zapisać student.

Każdy student może być opisany, podobnie jak zajęcia, czytelnym dla człowieka napisem (np. PESEL, imię, nazwisko).

Opis zajęć i grup

Opis zajęć i grup składa się z listy par, składających się z numeru zajęć i numeru grupy. Pojedyncza para `[numerZajęć, numerGrupy]` oznacza, że student jest zarejestrowany na zajęcia o numerze *numerZaj* do grupy o numerze *numerGrupy*. Program dopuszcza możliwość, że student jest zarejestrowany na zajęcia, ale nie jest zapisany z tych zajęć do żadnej grupy. W takim przypadku w pliku wejściowym powinna się pojawić para `[numerZajęć, -999]`.

Przykład: `[[100,1],[101,-999],[103,2]]` oznacza, że z zajęć 100 student jest zapisany do grupy 1, z zajęć 103 jest zapisany do grupy 2, a z zajęć 101 student nie jest zapisany do żadnej grupy!

Opis pakietów

Opis pakietów jest tablicą pojedynczych pakietów. Kolejność pakietów na liście nie odgrywa żadnej roli.

Pojedynczy pakiet to lista par `[numerZajęć, numerGrupy]`, na które student chce się zapisać. Przykładowo, jeśli student zapisał się na zajęcia `[101,102,103,104]`, to

pakiet `[[101,12],[102,22],[103,32]]` oznacza, że z zajęć 101 chce dostać się do grupy 12, z zajęć 102 do grupy 22, a z zajęć 3 do grupy 32. W jednym pakiecie nie mogą się powtarzać numery zajęć, ale nie jest konieczne aby występowały w nim wszystkie numery zajęć na które student się zapisał.

Pakiet zostanie przez program zrealizowany albo w całości albo wcale.

2.2.4 Przykładowy plik wejściowy

```
2 # dwa zajecia
# zajecia o zaj_cyk_id=105374, trzy grupy
105374 "1000-215aSOP,2008Z,CW"
# grupa 1, limit 15, poniedziałek 10:00-12:00 i sroda 10:00-12:00
# grupa 2, limit 10, poniedziałek 10:00-12:00
# grupa 3, limit 10, wtorek 12:00-13:00
[[1, 15, 0, [[1, 1000, 1200],[3,1000,1200]]],
 [2, 10, 0, [[1, 1000, 1200]]], [3, 10, 0, [[2, 1200, 1300]]]]
# zajecia o zaj_cyk_id=105386, trzy grupy
105386 "1000-225aSOP, 2008Z, LAB"
# grupa 1, limit 15, poniedziałek 10:00-12:00 i sroda 10:00-12:00
# grupa 2, limit 10, poniedziałek 10:00-12:00
# grupa 3, limit 10, wtorek 12:00-13:00
[[1, 15, 0, [[1, 1000, 1200],[3,1000,1200]]],
 [2, 10, 0, [[1, 1000, 1200]]], [3, 10, 0, [[2, 1200, 1300]]]]
3 # trzy grupy zwiazane
105374 1 105386 [1]
# zajecia o zaj_cyk_id=105374 grupa 1 pozwala chodzic na jedna grupe:
# zajecia 105386 grupa 1
105374 2 105386 [2]
# zajecia o zaj_cyk_id=105374 grupa 2 pozwala chodzic na jedna grupe:
# zajecia 105386 grupa 2
105374 3 105386 [3]
# zajecia o zaj_cyk_id=105374 grupa 3 pozwala chodzic na jedna grupe:
# zajecia 105386 grupa 3

# Część II, OSOBY
3 # trzy osoby
34267 "Jan Kowalski, 1234567890"
# osoba o os_id=34267 chodzi na zajecia 105374 do grupy 1, chce trafić
do grupy 2 lub 3
5 [[105374,1]] [[[105374,2]], [[105374,3]]]
467143 "Andrzej Nowak, 2345678901"
# osoba o os_id=467143 chodzi na zajecia 105386 do grupy 1, waga 5,
chce trafić do grupy 4
5 [[105386,1]] [[[105386,4]]]
88456 "Anna Kowalska, 456789012345"
```

```
# osoba o os_id=88456 i wadze 2 chodzi na zajecia 105374 i 105386 do grup 1,  
  chce zamienić obie te grupy równocześnie na grupy 2  
2 [[105374,1],[105386,1]] [[[105374,2],[105386,2]]]
```

2.3 Struktura pliku wyjściowego

Każdy z wierszy pliku opisuje jednego studenta. Opis pojedynczego studenta składa się z numeru tego studenta oraz tablicy trójek liczb (*z*, *gp*, *gk*), z których każda trójka oznacza, że na zajęcia *z* student powinien zostać przepisany z grupy *gp* (-999 jeśli nie był zapisany do żadnej grupy) do grupy *gk*.

2.3.1 Przykładowy plik wyjściowy

Poniżej przedstawiony jest plik wyjściowy generowany przez program dla przykładu z [2.2.4](#).

```
#KC (rev 100)  
34267 [[105374,1,2]]  
467143 [[105386,1,4]]  
88456 [[105374,1,2],[105386,1,2]]
```

2.4 Raport

Raport jest w formacie CSV i składa się z wierszy w których kolejne kolumny oznaczają:

- identyfikator studenta,
- liczba pakietów które zostały zrealizowane studentowi,
- liczba pakietów zdefiniowanych przez studenta.

Przykładowy raport:

```
34267,1,2  
467143,1,1  
88456,1,1
```

2.5 Dziennik

W trakcie swojego działania program generuje dziennik. W przeciwieństwie do RDG jest on o wiele bardziej opisowy i nie ma sztywnego formatu. Pierwsza jego część to powtórzenie wczytanych danych dla kontroli poprawności wejścia.

Druga część opisuje jakość rozwiązania oraz bieżące informacje z realizacji algorytmu hill-climbing w następującej postaci:

```
...  
Usuwa SID: 66978 zestaw: 0 przekroczen 1  
Usuwa SID: 66978 zestaw: 2 przekroczen 1  
Dodaje SID: 66978 zestaw: 3 przekroczen 0  
...
```

które oznaczają, że studentowi o numerze 66978 program anulował zestawy 0 i 2 a następnie zrealizował zestaw 3, który być może w dalszej części działania programu zostanie anulowany.

Rozdział 3

Opis działania programu

Działanie programu składa się z następujących kroków:

1. Wczytanie danych wejściowych. Dane wczytywane są do wektorów grupy i studenci. Pojedynczy obiekt Grupa jest odpowiednikiem jednej grupy z jednych zajęć, a Student jest odpowiednikiem studenta wraz z jego pakietami.
2. Wywołanie funkcji solve, która znajduje rozwiązanie.
3. Wypisanie rozwiązania.
4. Wygenerowanie raportu.

3.1 Sygnalizacja błędów

W przeciwieństwie do silnika RDG, giełda sygnalizuje błędy za pomocą asercji. Zdobyte doświadczenie uczy, że błędy można dużo szybciej poprawić w momencie, kiedy znamy ich dokładną lokalizację w kodzie a nie tylko ich semantykę. Spowalnia to co prawda program, ale w ramach przeprowadzonych testów i tak nigdy nie działał on dłużej niż kilka sekund, a więc to rozwiązanie wydaje się być lepsze.

3.2 Algorytm

Rozwiązaniem naszego problemu jest podzbiór, być może pusty, wszystkich pakietów zdefiniowanych przez użytkowników. Jeśli element jest w podzbiorze, to będziemy mówić, że realizujemy pakiet. Zwróćmy uwagę na to, że tylko niektóre podzbiory pakietów są sensowne. Jeśli ta sama osoba miałaby być przepisana do dwóch różnych grup z tych samych zajęć, to nazwiemy takie rozwiązanie *rozwiązaniem niedopuszczalnym*. Algorytm przeszukuje jedynie przestrzeń *rozwiązań dopuszczalnych*, czyli takich, które nie są rozwiązaniami niedopuszczalnymi. Zgodnie z naszą definicją rozwiązanie dopuszczalne może przekraczać limity, ale nasz algorytm w momencie stopu będzie generował jedynie rozwiązania nieprzekraczające limitów.

Algorytm, za pomocą techniki hill-climbing, szuka rozwiązania o jak najniższej wartości funkcji celu, na którą składają się dwa elementy:

- liczba pakietów, których realizacja powoduje przekroczenie limitu w grupie pomnożona przez stałą większą od jeden,
- minus liczba zrealizowanych pakietów.

Pojedynczy ruch algorytmu to zrealizowanie, bądź anulowanie zrealizowania pojedynczego pakietu. Zwróćmy uwagę na to, iż jeśli nie zostanie zrealizowany żaden pakiet to nie zostanie również przekroczony limit w żadnej grupie, oraz że jeśli limit w jakiejś grupie jest przekroczony, to istnieje taki ruch, który zmniejsza wartość funkcji celu (anulowanie któregoś pakietu), a więc w momencie stopu rozwiązanie nie będzie przekraczało limitów.

Algorytm działa w następujący sposób:

- Losowane jest maksymalne w sensie teoriomnogościowym rozwiązanie początkowe, które jest dopuszczalne. Rozwiązanie to być może przekracza limity.
- Za pomocą randomizowanego hill-climbingu rozwiązanie jest poprawiane.
- W momencie, których nie ma już możliwości poprawienia rozwiązania, program wypisuje wyniki.